Adatbázisrendszerek 2 BSc

7. gyakorlat

2026. 03. 25.

PL/SQL blokk, vezérlési szerkezet

Készítette: Kádár Balázs

Mérnökinformatikus Bsc: CFW04U

Miskolc, 2026

1. Írjon egy olyan PL/SQL programot, ami kiírja a kimenetre, hogy "Vezetéknév Keresztnév"! (VezKer.sql)

```
VezKer.sql > DBMS_OUTPUT.PUT_LINE
BEGIN
    DBMS_OUTPUT.PUT_LINE('Kádár Balázs');
END;
```

2. Írjon egy olyan PL/SQL programot, amely összead két számot és kiírja a kimenetre! (osszead.sql)

```
osszead.sql > ...
DECLARE
    a NUMBER := 10;
    b NUMBER := 5;
    osszeg NUMBER;
BEGIN
    osszeg := a + b;
    DBMS_OUTPUT.PUT_LINE('Az összeg: ' || osszeg);
END;
```

3. Írjon egy olyan PL/SQL programot, amely összeszoroz két számot és kiírja a kimenetre! (szorzas.sql)

```
szorzas.sql > ...
DECLARE
    a NUMBER := 12;
    b NUMBER := 6;
    szorzat NUMBER;
BEGIN
    szorzat := a * b;
    DBMS_OUTPUT.PUT_LINE('A szorzat: ' || szorzat);
END;
```

4. Írjon egy olyan PL/SQL programot, amely kiírja a "Vezetéknév Keresztnév" szöveget *csupa nagy* ill. *kis betűkkel* a kimenetére! (nagybetu.sql)

```
nagybetu.sql > ...
DECLARE
    szoveg VARCHAR2(100) := 'Kádár Balázs';
BEGIN
    DBMS_OUTPUT.PUT_LINE('Nagybetűvel: ' || UPPER(szoveg));
    DBMS_OUTPUT.PUT_LINE('Kisbetűvel: ' || LOWER(szoveg));
END;
```

5. Írjon egy olyan PL/SQL programot, amely összefűzve kiírja a kimenetére a "Vezetéknév " és a "Keresztnév" string-eket! (összefuz.sql)

```
osszefuz.sql > ...
DECLARE
    vezeteknev VARCHAR2(50) := 'Kádár';
    keresztnev VARCHAR2(50) := 'Balázs';
    teljes_nev VARCHAR2(100);
BEGIN
    teljes_nev := vezeteknev || ' ' || keresztnev;
    DBMS_OUTPUT.PUT_LINE(teljes_nev);
END;
```

6. Írjon egy olyan PL/SQL programot, amely kiírja az aktuális rendszeridőt! (sysdate.sql)

```
sysdate.sql > ...
BEGIN
    DBMS_OUTPUT.PUT_LINE('Aktuális rendszeridő: ' || SYSDATE);
END;
```

7. Írjon egy olyan PL/SQL programot, amely kiírja a rendszeridőt 'YYYY-MM-DD' formátumban! (sysdate1.sql)

```
sysdate1.sql > ...
BEGIN
    DBMS_OUTPUT.PUT_LINE('Dátum: ' || TO_CHAR(SYSDATE, 'YYYY-MM-DD'));
END;
```

8. Számítsa ki a kör területét, ha az r=12. (kor.sql)

```
kor.sql > ...
DECLARE
    r NUMBER := 12;
BEGIN
    DBMS_OUTPUT.PUT_LINE('A kör területe: ' || (r * r * 3.14));
END;
```

Vezérlési szerkezet

2. Írjon egy olyan PL/SQL programot, amely eldönti két szám közül, hogy melyik a nagyobb (a=10; b=81)! (vezerles2.sql)

```
vezerles2.sql > ...
DECLARE
    a NUMBER := 14;
    b NUMBER := 61;
BEGIN
    IF a > b THEN
        DBMS_OUTPUT.PUT_LINE('A nagyobb szám: ' || a);
    ELSIF b > a THEN
        DBMS_OUTPUT.PUT_LINE('A nagyobb szám: ' || b);
    ELSE
        DBMS_OUTPUT.PUT_LINE('A két szám egyenlő.');
    END IF;
END;
```

3. Írjon egy olyan PL/SQL programot, amely eldönti egy számról, hogy bele esik e két másik szám által megadott intervallumba! (vezerles3.sql)

```
vezerles3.sql > ...
DECLARE
    min_ertek NUMBER := 10;
    max_ertek NUMBER := 100;
    ertek NUMBER := 78;
BEGIN
    IF ertek >= min_ertek AND ertek <= max_ertek THEN
        DBMS_OUTPUT.PUT_LINE('Az érték beleesik az intervallumba.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Az érték nem esik bele az intervallumba.');
    END IF;
END;
```

4. Ismertesse a Swith Case szerkezetet!

A CASE szerkezet lehetővé teszi, hogy egy változó vagy kifejezés értéke alapján több lehetséges végrehajtási ág közül válasszunk. Tisztább és olvashatóbb, mint a sok egymásba ágyazott IF-THEN-ELSIF.

Két fő típusa van:

1. ***Egyszerű CASE (simple)***

   pl:
   ```
   CASE v_auto_szin
     WHEN 'piros' THEN
       DBMS_OUTPUT.PUT_LINE('Sportos választás.');
     WHEN 'fekete' THEN
       DBMS_OUTPUT.PUT_LINE('Elegáns megjelenés.');
     ELSE
       DBMS_OUTPUT.PUT_LINE('Általános szín.');
   END CASE;
   ```

2. ***Kereső CASE (Searched CASE)***

   pl:
   ```
   CASE
     WHEN v_ar > 5000000 THEN
       DBMS_OUTPUT.PUT_LINE('Luxus kategória');
     WHEN v_ar BETWEEN 1000000 AND 5000000 THEN
       DBMS_OUTPUT.PUT_LINE('Középkategória');
     ELSE
       DBMS_OUTPUT.PUT_LINE('Olcsó autó');
   END CASE;
   ```

5. Írjon egy olyan PL/SQL programot, amely a beosztás rövidítése alapján megadja, a dolgozó teljes beosztását! (vezerles4.sql) Pl.: beosztas = 'root', ez alapján kiírja: Rendszergazda (UNIX/LINUX).

vezerles4.sql > ...

```sql
DECLARE
    beosztas VARCHAR2(20) := 'root';
BEGIN
    CASE beosztas
        WHEN 'root' THEN
            DBMS_OUTPUT.PUT_LINE('Rendszergazda (UNIX/LINUX)');
        WHEN 'dba' THEN
            DBMS_OUTPUT.PUT_LINE('Adatbázis-adminisztrátor');
        WHEN 'dev' THEN
            DBMS_OUTPUT.PUT_LINE('Fejlesztő');
        WHEN 'hr' THEN
            DBMS_OUTPUT.PUT_LINE('Humán erőforrás munkatárs');
        ELSE
            DBMS_OUTPUT.PUT_LINE('Ismeretlen beosztás');
    END CASE;
END;
```

6. Írjon egy olyan PL/SQL programot, amely eldönti három számról, hogy alkothat e háromszöget! (vezerles5.sql)

a=3; b=4; c=5;

vezerles5.sql > ...

```sql
DECLARE
    a NUMBER := 3;
    b NUMBER := 4;
    c NUMBER := 5;
BEGIN
    IF a + b > c AND a + c > b AND b + c > a THEN
        DBMS_OUTPUT.PUT_LINE('A három szám alkothat háromszöget.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('A három szám nem alkothat háromszöget.');
    END IF;
END;
```

7. Írjon egy olyan PL/SQL programot, amely a háromszög három oldala ismeretében kiszámolja a háromszö területét a Héron képlet segítségével! (vezerles6.sql)

a=3; b=4; c=5;

Heron képlet:

$$T = \sqrt{s(s-a)(s-b)(s-c)}$$

$$s = \frac{a+b+c}{2}$$

ahol *a*, *b* és *c* a háromszög oldalai, *s* a háromszög kerületének a fele, és *T* a háromszög területe.

vezerles6.sql > ...

```
DECLARE
    a NUMBER := 3;
    b NUMBER := 4;
    c NUMBER := 5;
    s NUMBER;
    t NUMBER;
BEGIN
    s := (a + b + c) / 2;
    t := SQRT(s * (s - a) * (s - b) * (s - c));

    DBMS_OUTPUT.PUT_LINE('A háromszög félkerülete: ' || s);
    DBMS_OUTPUT.PUT_LINE('A háromszög területe: ' || t);
END;
```

8. Ismertesse a Loop, While és a For ciklust PL/SQL-ben!

Az Alapvető ciklus (LOOP)

Ez a legegyszerűbb ciklus. Nincs előre meghatározott feltétele, így addig fut, amíg egy belső utasítással ki nem kényszerítjük a kilépést.

Jellemzője: Legalább egyszer biztosan lefut.

Kilépés: Az EXIT vagy EXIT WHEN kulcsszóval.

pl:

```
LOOP
  v_szamlalo := v_szamlalo + 1;
  EXIT WHEN v_szamlalo > 10; -- Kilépési feltétel
END LOOP;
```

A WHILE ciklus

A ciklusmag lefutása előtt ellenőrzi a feltételt. Ha a feltétel már az elején hamis, a ciklus egyszer sem fut le.

Jellemzője: Akkor használjuk, ha nem tudjuk pontosan, hányszor kell ismételni, de a futás feltételét ismerjük.

pl:

```
WHILE v_szamlalo <= 10 LOOP
   v_szamlalo := v_szamlalo + 1;
END LOOP;
```

3. A FOR ciklus (Számláló ciklus)

Egy előre meghatározott tartományon (alsó és felső határ között) fut végig.

Jellemzője: Automatikusan kezeli a számláló változót (nem kell deklarálni a DECLARE részben). Alapértelmezetten egyesével növeli az értéket.

Visszafelé: A REVERSE kulcsszóval fordított sorrendben is futtatható.

pl:

```
FOR i IN 1..10 LOOP
   DBMS_OUTPUT.PUT_LINE('Érték: ' || i);
END LOOP;
```

9. Írjon egy olyan PL/SQL programot, amely kiírja 1-től n-ig (n=10) a számokat! (vezerles7.sql)

vezerles7.sql > ...

```
1  DECLARE
2      n NUMBER := 20;
3  BEGIN
4      FOR i IN 1..n LOOP
5          DBMS_OUTPUT.PUT_LINE(i);
6      END LOOP;
7  END;
```

10. Írjon egy olyan PL/SQL programot, amely kiírja az első n darab Fibonacci számot! (vezerles8.sql

vezerles8.sql > ...

```
DECLARE
    n NUMBER := 10;
    a NUMBER := 0;
    b NUMBER := 1;
    c NUMBER;
BEGIN
    DBMS_OUTPUT.PUT_LINE('Fibonacci sorozat:');

    FOR i IN 1..n LOOP
        DBMS_OUTPUT.PUT_LINE(a);
        c := a + b;
        a := b;
        b := c;
    END LOOP;
END;
```

11. Írjon egy olyan PL/SQL programot, amely eldönti egy n számról, hogy prímszám-e! (vezerles9.sql) (n=10)

vezerles9.sql > ...

```
DECLARE
    n NUMBER := 10;
    prim BOOLEAN := TRUE;
BEGIN
    IF n < 2 THEN
        prim := FALSE;
    ELSE
        FOR i IN 2..TRUNC(SQRT(n)) LOOP
            IF MOD(n, i) = 0 THEN
                prim := FALSE;
                EXIT;
            END IF;
        END LOOP;
    END IF;

    IF prim THEN
        DBMS_OUTPUT.PUT_LINE(n || ' prímszám.');
    ELSE
        DBMS_OUTPUT.PUT_LINE(n || ' nem prímszám.');
    END IF;
END;
```

1. A relációs séma alapján készítse el a következő táblát:

Mentés: **neptunkod_*auto.sql***

Hozza létre egy *auto* nevű táblát a *PLSQL fejlesztővel*. **Mentés:** *auto.sql*

Auto [rsz PK, tipus, szin, kor, ar]

CFW04U_auto.sql > ...

```sql
CREATE TABLE auto (
    rsz VARCHAR2(10) PRIMARY KEY,
    tipus VARCHAR2(50),
    szin VARCHAR2(30),
    kor NUMBER(3),
    ar NUMBER(12, 2)
);
```

2. Írjon egy PL/SQL blokkot egy autó rekord felvitelére, ahol a mezőértékeket memóriaváltozókon keresztül adjuk meg, melyek típusa az AUTO tábla mezőinek típusával megegyező. **Mentés:** *autos_feltoltegysor.sql*

auto				
rsz	tipus	szin	kor	ar

autos_feltoltegysor.sql > ...

```sql
DECLARE
    v_rsz   auto.rsz%TYPE   := 'ABC-123';
    v_tipus auto.tipus%TYPE := 'Opel Astra';
    v_szin  auto.szin%TYPE  := 'Ezüst';
    v_kor   auto.kor%TYPE   := 5;
    v_ar    auto.ar%TYPE    := 3500000;
BEGIN
    INSERT INTO auto (rsz, tipus, szin, kor, ar)
    VALUES (v_rsz, v_tipus, v_szin, v_kor, v_ar);

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('Az autó sikeresen felvéve a rendszerbe!');
EXCEPTION
    WHEN DUP_VAL_ON_INDEX THEN
        DBMS_OUTPUT.PUT_LINE('Hiba: Ez a rendszám már létezik!');
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Váratlan hiba történt!');
        ROLLBACK;
END;
```

3. Töltse fel a táblát a minta alapján. **Mentés:** *autos_feltolt.sql*

	RSZ	TIPUS	SZIN	KOR	AR
1	ABC500	Opel Corsa	piros	8	720000
2	MCC325	Opel Insignia	fekete	2	6800000
3	BBM104	Suzuki Swift	piros	5	1500000
4	CHR411	Renault Twingo	piros	12	630000
5	JRZ932	BMW M3	fekete	5	4500000
6	JEG113	Opel Corsa	piros	7	810000
7	DER842	Seat Ibiza	szürke	14	450000
8	BAB422	Lada 1300S	fehér	28	198000
9	UFF666	Audi TT	fekete	4	7500000
10	ABC124	Fiat	Piros	3	18000000

autos_feltolt.sql > ...

```sql
INSERT INTO auto (rsz, tipus, szin, kor, ar) VALUES ('ABC500', 'Opel Corsa', 'piros', 8, 720000);
INSERT INTO auto (rsz, tipus, szin, kor, ar) VALUES ('MCC325', 'Opel Insignia', 'fekete', 2, 6800000);
INSERT INTO auto (rsz, tipus, szin, kor, ar) VALUES ('BBM104', 'Suzuki Swift', 'piros', 5, 1500000);
INSERT INTO auto (rsz, tipus, szin, kor, ar) VALUES ('CHR411', 'Renault Twingo', 'piros', 12, 630000);
INSERT INTO auto (rsz, tipus, szin, kor, ar) VALUES ('JRZ932', 'BMW M3', 'fekete', 5, 4500000);
INSERT INTO auto (rsz, tipus, szin, kor, ar) VALUES ('JEG113', 'Opel Corsa', 'piros', 7, 810000);
INSERT INTO auto (rsz, tipus, szin, kor, ar) VALUES ('DER842', 'Seat Ibiza', 'szürke', 14, 450000);
INSERT INTO auto (rsz, tipus, szin, kor, ar) VALUES ('BAB422', 'Lada 1300S', 'fehér', 28, 198000);
INSERT INTO auto (rsz, tipus, szin, kor, ar) VALUES ('UFF666', 'Audi TT', 'fekete', 4, 7500000);
INSERT INTO auto (rsz, tipus, szin, kor, ar) VALUES ('ABC124', 'Fiat', 'Piros', 3, 18000000);

COMMIT;
```

4. Írjon egy PL/SQL blokk-t, mely egy memóriaváltozóban megadott kornál idősebb autók (7 év) árát csökkenti 10%-al. **Mentés:** *auto_ar_csokk_10.sql*

auto_ar_csokk_10.sql > ...

```sql
SET SERVEROUTPUT ON;

DECLARE
    v_kor_hatar auto.kor%TYPE := 7;
    v_sorok_szama NUMBER;
BEGIN
    UPDATE auto
    SET ar = ar * 0.9
    WHERE kor > v_kor_hatar;

    v_sorok_szama := SQL%ROWCOUNT;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE(v_sorok_szama || ' darab 7 évnél idősebb autó ára csökkentve 10%-kal.');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Hiba történt a frissítés során!');
        ROLLBACK;
END;
```

5. Írjon egy PL/SQL blokk-t az autók átlagárának lekérdezésére és kiíratása.

Mentés: *auto_avg_dbms_kiir.sql*

auto_avg_dbms_kiir.sql > ...

```sql
SET SERVEROUTPUT ON;

DECLARE
    v_atlag_ar NUMBER(12, 2);
BEGIN
    SELECT AVG(ar)
    INTO v_atlag_ar
    FROM auto;

    IF v_atlag_ar IS NOT NULL THEN
        DBMS_OUTPUT.PUT_LINE('Az autók átlagos ára: ' || ROUND(v_atlag_ar, 0) || ' Ft');
    ELSE
        DBMS_OUTPUT.PUT_LINE('A tábla üres, nem számolható átlagár.');
    END IF;
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Hiba történt a lekérdezés során!');
END;
```

6. Írjon egy PL/SQL blokk-t, amely a memóriaváltozóban 10 évnél idősebb auto törlésére.
Mentés: *autos_kor_torles.sql*

autos_kor_torles.sql > ...

```sql
SET SERVEROUTPUT ON;

DECLARE
    v_limit_kor NUMBER := 10;
    v_torolt_db NUMBER;
BEGIN
    DELETE FROM auto
    WHERE kor > v_limit_kor;

    v_torolt_db := SQL%ROWCOUNT;

    COMMIT;

    IF v_torolt_db > 0 THEN
        DBMS_OUTPUT.PUT_LINE('Sikeresen törölve lett ' || v_torolt_db || ' darab 10 évnél idősebb autó.');
    ELSE
        DBMS_OUTPUT.PUT_LINE('Nem találtam 10 évnél idősebb autót a táblában.');
    END IF;

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Hiba történt a törlés során!');
        ROLLBACK;
END;
```

7. Írjon egy PL/SQL blokk-t az *autók darabszámának lekérdezésére* és kiíratása!

Mentés: *(auto_db_lekerd.sql)*

auto_db_lekerd.sql > ...

```sql
SET SERVEROUTPUT ON;

DECLARE
    v_autok_szama NUMBER;
BEGIN
    SELECT COUNT(*)
    INTO v_autok_szama
    FROM auto;

    DBMS_OUTPUT.PUT_LINE('Az adatbázisban jelenleg tárolt autók száma: ' || v_autok_szama || ' db');

EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Hiba történt a darabszám lekérdezése során!');
END;
```

8. Készítsen egy tárolt eljárás a 10 évnél idősebb autók törlésére!

Mentés: *(auto_torlese_10_nagyobb.sql)*

Tárolt eljárás hívás: *auto_torol_elj.sql*

auto_torlese_10_nagyobb.sql > ...

```sql
CREATE OR REPLACE PROCEDURE auto_torlese_10_nagyobb AS
BEGIN
    DELETE FROM auto
    WHERE kor > 10;

    COMMIT;

    DBMS_OUTPUT.PUT_LINE('A 10 évnél idősebb autók törlése sikeresen megtörtént.');
EXCEPTION
    WHEN OTHERS THEN
        DBMS_OUTPUT.PUT_LINE('Hiba történt a törlés során!');
        ROLLBACK;
END;
```